

P2615R1: Meaningful exports

Audience: EWG, CWG

S. Davis Herring <herring@lanl.gov>

November 11, 2022

History

Since [r0](#):

- Removed outdated change to `[basic.def]`
- Allowed bare export of certain declarations requested by EWG (and *deduction-guides*, whose status is unclear, for the sake of discussion)

Introduction

This paper addresses [CWG2443](#) by forbidding applying export directly to certain meaningless declarations. Conversely, it also allows all kinds to appear with `export {}`. For consistency extern "C" is given the same restrictions when used directly.

Approach

The expansion of permissions here was originally proposed in [P1766](#), which SG2 considered in Cologne but which has never been seen by EWG (perhaps because the rest of it was considered “accepted”). EWG did [see the corresponding NB comment](#), but the vote was too close to progress it. No issues have surfaced with the general direction, but the wording does not specify a coherent set of restrictions; as such, this wording makes most of the change (largely by separating the relevant candidates in the *declaration* grammar). EWG has seen this proposal several times, with varied outcomes; the most recent (in Kona 2022) requested to allow certain declarations unlikely to mislead programmers into thinking they have some effect.

Wording

Relative to N4917.

#[basic]

#[basic.pre]

Change paragraph 5:

- Every name is introduced by a *declaration*, which is a

1. **name**-*declaration*, [...]

#[basic.lookup.unqual]

Change paragraph 1:

- [...]

~~[Note: A using-directive is exported if and only if it appears in a header unit. — end note]~~

#[dcl.dcl]

#[dcl.pre]

Change the grammar in paragraph 1:

- [...]

declaration:

name-declaration
special-declaration

name-declaration:

block-declaration
nodeclspec-function-declaration
function-definition
template-declaration
deduction-guide
~~*explicit-instantiation*~~
~~*explicit-specialization*~~
~~*export-declaration*~~
linkage-specification
namespace-definition
empty-declaration
attribute-declaration
module-import-declaration

special-declaration:

explicit-instantiation
explicit-specialization
export-declaration

block-declaration:

simple-declaration
asm-declaration
namespace-alias-definition
using-declaration
using-enum-declaration
using-directive
static_assert-declaration
alias-declaration
opaque-enum-declaration

[...]

#[dcl.link]

Change the grammar in paragraph 2:

- *linkage-specification:*

extern string-literal { *declaration-seq*_{opt} }
extern string-literal *name-declaration*

Change paragraph 4:

- ~~A module-import-declaration shall not be directly contained in a linkage-specification.~~ A module-import-declaration appearing in a linkage specification with other than C++ language linkage is conditionally-supported with implementation-defined semantics.

#[module.interface]

Change the grammar before paragraph 1:

- *export-declaration*:

```
export name-declaration
export { declaration-seqopt }
export-keyword module-import-declaration
```

Change paragraph 1:

- An *export-declaration* shall inhabit a namespace scope and appear in the purview of a module interface unit. An *export-declaration* shall not appear directly or indirectly within an unnamed namespace or a *private-module-fragment*. An *export-declaration* has the declarative effects of its **name**-*declaration*, *declaration-seq* (if any), or *module-import-declaration*. **The name-declaration of an export-declaration shall not declare a partial specialization ([temp.decls.general]).** The ~~declaration or~~ *declaration-seq* of an *export-declaration* shall not contain an *export-declaration* or *module-import-declaration*.

[Note: An *export-declaration* does not establish a scope. — end note]

Change paragraph 3:

- ~~An exported declaration that is not a module-import-declaration shall declare at least one name.~~ If the **an exported** declaration is not within a header unit, it shall not declare a name with internal linkage.

Change paragraph 4:

- [...]

```
export module M;
export namespace {}           // error: does not introduce any names namespace has internal linkage
export namespace {
int a1;                      // error: export of name with internal linkage
}
namespace {
    export int a2;             // error: export of name with internal linkage
}
export static int b;           // error: b explicitly declared static
export int f();                // OK
export namespace N { }        // OK
export using namespace N;      // error: does not declare a name OK
```

[...]

#[temp]

#[temp.expl.spec]

Change paragraph 2:

- **The declaration in an explicit-specialization shall not be an export-declaration.** An explicit specialization shall not use a *storage-class-specifier* ([dcl.stc]) other than `thread_local`.